

이산수학

9월 22일 수업자료

소프트웨어학부
김형균 교수

행렬식

퀴즈

n 차 정방행렬에서 i 행과 j 열을 제거해서 만든 $(n-1)$ 차 정방행렬은 무엇인가?

프로그래밍 실습

예제

n x m 행렬의 역행렬을 구하는 프로그램을 작성하고 다음 3 x 3 행렬의 역행렬을 구하라.

$$\begin{bmatrix} 2 & 3 & -1 \\ 4 & 4 & -3 \\ -2 & 3 & -1 \end{bmatrix}$$

행렬을 예쁘게 출력

```
def matrixout(mx, size):
    print("┌" + " " * size + "┐")
    for i in range(size):
        print("|", end=" ")
        for j in range(size):
            print("%7.3f" % mx[i][j], end=" ")
        print("|")
    print("└" + " " * size + "┘")
```

전치 행렬 구하기

```
def transposeMatrix(m):
    return [[m[j][i] for j in range(len(m))] for i in range(len(m[0]))]
```

소행렬 구하기

```
def getMatrixMinor(m, i, j):
    return [row[:j] + row[j+1:] for row in (m[:i] + m[i+1:])]
```

행렬식 계산 (재귀)

```
def getMatrixDeterminant(m):
    if len(m) == 1:
        return m[0][0]
    if len(m) == 2:
        return m[0][0]*m[1][1] - m[0][1]*m[1][0]

    determinant = 0
    for c in range(len(m)):
        determinant += ((-1) ** c) * m[0][c] * getMatrixDeterminant(getMatrixMinor(m, 0, c))
    return determinant
```

역행렬이 존재하는지 확인

```
def has_inverse(m):
    determinant = getMatrixDeterminant(m)
    if abs(determinant) < 1e-10: # 부동소수점 오차 고려
        print('오류! 해당 행렬의 행렬식은 0입니다.')
        print('역행렬 계산은 불가능합니다!')
        return False
    return True
```

프로그래밍 실습

```
# 역행렬 계산
def getMatrixInverse(m):
    determinant = getMatrixDeterminant(m)

    if len(m) == 1:
        return [[1.0 / m[0][0]]]

    if len(m) == 2:
        return [[m[1][1] / determinant, -1 * m[0][1] / determinant],
                [-1 * m[1][0] / determinant, m[0][0] / determinant]]

    cofactors = []
    for r in range(len(m)):
        cofactorRow = []
        for c in range(len(m)):
            minor = getMatrixMinor(m, r, c)
            cofactorRow.append((-1) ** (r + c) * getMatrixDeterminant(minor))
        cofactors.append(cofactorRow)

    adjugate = transposeMatrix(cofactors)

    for r in range(len(adjugate)):
        for c in range(len(adjugate)):
            adjugate[r][c] = adjugate[r][c] / determinant

    return adjugate
```

```
# ===== 메인 실행부 =====
def main():
    try:
        k = int(input("정방행렬의 차수를 입력하십시오: "))
        if k <= 0:
            print("차수는 양의 정수여야 합니다.")
            return

        print(f"{k}x{k} 정방행렬 A를 입력하세요 (각 행을 공백으로 구분하여 한 줄씩 입력):")
        matrix_a = [] # 변수명을 a에서 matrix_a로 변경
        for i in range(k):
            while True:
                try:
                    row_input = input(f"{i+1}행: ").strip()
                    # list() 함수 대신 리스트 컴프리헨션 사용
                    row_values = [float(x) for x in row_input.split()]
                    if len(row_values) != k:
                        print(f"입력 오류: 정확히 {k}개의 값을 입력해야 합니다.")
                        continue
                    matrix_a.append(row_values)
                    break
                except ValueError:
                    print("입력 오류: 숫자만 입력해주세요.")

        if has_inverse(matrix_a): # 변수명 변경에 맞춰 수정
            print("\n해당 행렬의 역행렬은 다음과 같습니다:")
            inverse = getMatrixInverse(matrix_a)
            matrixout(inverse, k)
        except ValueError:
            print("입력 오류: 정수를 입력해주세요.")
        except Exception as e:
            print(f"예상치 못한 오류가 발생했습니다: {e}")

    print()

# 프로그램 시작점
if __name__ == "__main__":
    main()
```

2.5 연립일차방정식과 역행렬



연립일차방정식

정의 2-17 일차방정식(선형방정식)

$a_1, a_2, \dots, a_n, b$ 가 실수일 때, 다음과 같이 표현되는 방정식을 일차방정식^{linear equation} 또는 선형방정식이라고 한다.

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = b$$

이때 $a_1, a_2, \dots, a_n$ 은 계수, b 는 상수이고 $x_1, x_2, \dots, x_n$ 은 미지수이다.

연립일차방정식

정의 2-18 연립일차방정식

m 개의 n 원일차방정식들로 구성된 방정식을 연립일차방정식(system of linear equations)이라고 한다.

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

연립일차방정식

- ✓ 연립일차방정식의 해를 구할 때 행렬을 이용하는 방법

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \quad \mathbf{X} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix} \text{라 하면}$$

- ✓ 연립일차방정식 $\mathbf{AX}=\mathbf{B}$ 는 다음과 같다.

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix} \text{----- (1)}$$

- ✓ 이때 행렬 $\mathbf{A}$ 를 ($m \times n$) **계수행렬**^{coefficient matrix}, 행렬 $\mathbf{X}$ 를 ($n \times 1$) **미지수행렬**^{unknown matrix}, 행렬 $\mathbf{B}$ 를 ($m \times 1$) **상수행렬**^{constant matrix}
- ✓ (1)번식을 풀어서 표현하면 [정의 2-18]과 같다.

연립일차방정식

예제 2-36

다음 연립일차방정식을 계수행렬, 미지수행렬, 상수행렬로 표현하라.

$$\begin{cases} 2x_1 + 3x_2 + 5x_3 = 7 & \text{--- ①} \\ 4x_1 + 5x_2 + 6x_3 = 6 & \text{--- ②} \\ 8x_1 + 7x_2 + 8x_3 = 5 & \text{--- ③} \end{cases}$$

연립일차방정식

❖ 가감법 : 직접 미지수를 제거해서 해를 구하는 방법

예제 2-37

다음 연립일차방정식을 가감법을 이용하여 해를 구하라.

$$\begin{cases} 2x_1 + 3x_2 + 5x_3 = 7 & \text{--- ①} \\ 4x_1 + 5x_2 + 6x_3 = 6 & \text{--- ②} \\ 8x_1 + 7x_2 + 8x_3 = 5 & \text{--- ③} \end{cases}$$

연립일차방정식

정의 2-19 확대행렬

계수행렬 A 의 오른쪽에 상수행렬을 추가하여 만든 행렬을 **확대행렬** augmented matrix이라고 한다.

❖ [정의 2-18]에 있는 연립일차방정식의 계수행렬 A 와 상수행렬 B 를 확대행렬로 표현하면 다음과 같다.

$$\checkmark \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right)$$

연립일차방정식

예제 2-38

다음 연립일차방정식을 확대행렬 형태로 바꾸시오

$$\begin{cases} 3x_2 + 4x_3 + 5x_4 = 6 \\ 3x_1 - 4x_2 + 5x_3 - 6x_4 = 7 \\ 4x_1 + 5x_2 \quad \quad + 7x_4 = 8 \\ 5x_1 \quad \quad + 7x_3 - 8x_4 = 9 \end{cases}$$

연립일차방정식

실습문제

다음 확대행렬을 연립일차방정식으로 표현하라.

$$\left(\begin{array}{cccc|c} 1 & 2 & 3 & 4 & 5 \\ 3 & 5 & 7 & 0 & 11 \\ 5 & 6 & 7 & 8 & 9 \\ 2 & 0 & 4 & 6 & 10 \end{array} \right)$$

연립일차방정식

❖ 가우스 소거법

- 연립방정식을 푸는 방법으로 정방행렬을 가우스 행렬로 변환하고, 가우스 행렬로부터 후진 대입법을 이용하여 연립방정식의 해를 구하는 방법

❖ 가우스-조던 소거법

- 연립방정식을 푸는 방법으로 가우스 소거법을 좀 더 개선한 방법이고, 확대행렬과 행 연산을 이용하여 역행렬을 구할 때도 사용한다.
- 가우스 소거법에서 연산을 조금 더 진행하면 가우스-조던 소거법이기 때문에 두 방식은 매우 비슷하다.

연립일차방정식

❖ 가우스 소거법

- ✓ 연립일차방정식을 확대행렬로 만들어 [알고리즘 2-5]에 있는 3가지 기본 행 연산을 반복해서 사용하여 가우스 행렬로 만든다. 그런 다음 후진 대입법을 이용하여 해를 구하는 방법이다.

정의 2-20 가우스 행렬

계수행렬의 주대각 성분을 모두 1로 만들면서, 주대각 성분의 아래쪽에 있는 모든 성분이 0이 되도록 하고, 주대각 성분의 위쪽에 있는 모든 성분은 계수들로 남겨놓은 형태의 확대행렬을 **가우스 행렬** Gauss matrix이라고 한다.

$$\left(\begin{array}{cccc|c} 1 & a_{12} & \cdots & a_{1n} & c_1 \\ 0 & 1 & \cdots & a_{2n} & c_2 \\ \vdots & & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & c_m \end{array} \right)$$

[알고리즘 2-5] 기본 행 연산

1. 두 행을 교환한다.
2. 한 행에 0이 아닌 스칼라곱을 한다.
3. 스칼라곱을 한 행과 다른 행을 더해 성분을 0으로 만든다.

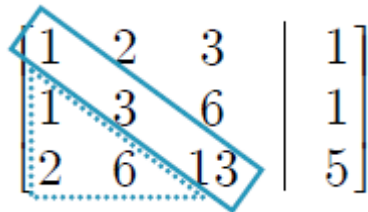
연립일차방정식

❖ 가우스 소거법

– 연립1차방정식의 해를 가우스 소거법으로 구해보기

$$\begin{cases} x + 2y + 3z = 1 \\ x + 3y + 6z = 1 \\ 2x + 6y + 13z = 5 \end{cases} \Rightarrow \left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & 1 & 3 \end{array} \right] \Rightarrow \begin{cases} x + 2y + 3z = 1 \\ y + 3z = 0 \\ z = 3 \end{cases}$$

- 위의 연립1차방정식을 첨가행렬 형태로 작성
- 첨가행렬의 형태를 가우스 행렬의 형태로 만들기 위해서는 실선 사각형 부분은 모두 1로, 점선 삼각형 부분은 모두 0으로 만듦


$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 1 & 3 & 6 & 1 \\ 2 & 6 & 13 & 5 \end{array} \right]$$

The diagram shows an augmented matrix with three rows and four columns. A solid blue triangle is drawn over the first three rows and columns, with its hypotenuse passing through the element 1 in the first row, first column. A dotted blue triangle is drawn over the last three rows and columns, with its hypotenuse passing through the element 1 in the second row, first column.

- (1) 첨가행렬의 1행 1열의 원소는 이미 1이므로 이 원소를 이용해 2행 1열의 원소와 3행 1열의 원소를 0으로 만듦

연립일차방정식

❖ 가우스 소거법

❶ 1행 $\times(-1)+2$ 행

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 1 & 3 & 6 & 1 \\ 2 & 6 & 13 & 5 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} -1 & -2 & -3 & -1 \\ 1 & 3 & 6 & 1 \\ 2 & 6 & 13 & 5 \end{array} \right] \Rightarrow \begin{array}{c} \text{원래대로 작성} \\ \left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 2 & 6 & 13 & 5 \end{array} \right] \end{array}$$

❷ 1행 $\times(-2)+3$ 행

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 2 & 6 & 13 & 5 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} -2 & -4 & -6 & -2 \\ 0 & 1 & 3 & 0 \\ 2 & 6 & 13 & 5 \end{array} \right] \Rightarrow \begin{array}{c} \text{원래대로 작성} \\ \left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 0 & 2 & 7 & 3 \end{array} \right] \end{array}$$

(2) ❷ 단계까지 구한 첨가행렬의 결과에서 2행 1열의 원소는 0, 2열의 원소는 1이므로 이 원소들을 이용해 3행 1열과 2열의 원소를 0, 3열의 원소를 1로 만듦

❸ 2행 $\times(-2)+3$ 행

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 0 & 2 & 7 & 3 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & -2 & -6 & 0 \\ 0 & 2 & 7 & 3 \end{array} \right] \Rightarrow \begin{array}{c} \text{원래대로 작성} \\ \left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & 1 & 3 \end{array} \right] \end{array}$$

연립일차방정식

❖ 가우스 소거법

- ③ 단계까지 수행 결과로 나오는 첨가행렬의 형태가 가우스 행렬의 형태임

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & 1 & 3 \end{array} \right]$$

위의 첨가행렬에서 계수행렬은 $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}$, 상수행렬은 $\begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}$, 미지수행렬은 $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ 임을 알 수 있

으므로 이를 이용해 다시 연립1차방정식을 작성하면 다음과 같다.

$$\begin{cases} x + 2y + 3z = 1 \\ y + 3z = 0 \\ z = 3 \end{cases}$$

- 위의 결과로 후진대입법으로 풀어보기

$z = 3$ 이므로 $y + 3z = 0$ 에 대입하면,

$$y + 3 \cdot 3 = y + 9 = 0 \quad \therefore y = -9$$

$y = -9$, $z = 3$ 이므로 $x + 2y + 3z = 1$ 에 대입하면,

$$x + 2 \cdot (-9) + 3 \cdot 3 = 1 \quad \therefore x = 10$$

그러므로 $x = 10$, $y = -9$, $z = 3$ 이다.